SIMATS ENGINEERING

CSA4712 – DEEP LEARNING

CO2 – ASSESSMENT 2

DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

Student Name	B Akshaya
Registration Number	192424336
Submission Date	29.10.2026
Scenario	Smart agriculture and crop disease detection

Assignment overview

This assignment explains how deep neural networks are designed, trained and analysed for a smart-agriculture centre: a feed-forward network predicts crop yield from rainfall, temperature, soil moisture and fertilizer quantity, and a convolutional network classifies healthy and diseased leaf images. Each task is presented as connected technical discussion with a worked calculation or data table, a code excerpt and a notebook-style output panel. The Keras excerpts are templates to run in Google Collab; the panels were produced by an equivalent NumPy implementation. Panels for Tasks 1 to 5 and 9 to 11 are computed directly from the numbers in the questions or from the architecture; panels for Tasks 6, 7, 8 and 12 are clearly labelled simulations on synthetic data, not field measurements. All outputs are decision support for agronomists.

TASK 1 — Deep feed-forward network architecture

(a) The four predictors (rainfall, temperature, soil moisture, fertilizer quantity) form the **input layer of 4 neurons**; the input neurons only hold the normalized values. **Hidden layer 1 has 8 neurons** and **hidden layer 2 has 4 neurons**, both with ReLU, so that the network can learn nonlinear interactions such as rainfall × fertilizer. Crop yield is one continuous number, so the **output layer has 1 neuron with a linear activation**. The architecture is 4 → 8 → 4 → 1. **(b)** In column-vector notation layer l has weight matrix $W^{(l)}$ of shape $n_l \times n_{l-1}$ and bias vector $b^{(l)}$ of length n_l: $W^{(1)}$ is 8×4 and $b^{(1)}$ is 8×1; $W^{(2)}$ is 4×8 and $b^{(2)}$ is 4×1; $W^{(3)}$ is 1×4 and $b^{(3)}$ is 1×1. This gives $32 + 8 + 32 + 4 + 4 + 1 = 81$ trainable parameters. **(c)** The normalized input vector x (4×1) is multiplied by $W^{(1)}$, the bias is added and ReLU is applied: $h^{(1)} = \max(0, W^{(1)}x + b^{(1)})$. The 8 activations go through the same operation with $W^{(2)}$ and $b^{(2)}$ to give $h^{(2)}$, and the output layer computes $\hat{y} = W^{(3)}h^{(2)} + b^{(3)}$. Information moves only forward, with no loops, which is why the model is called feed-forward.

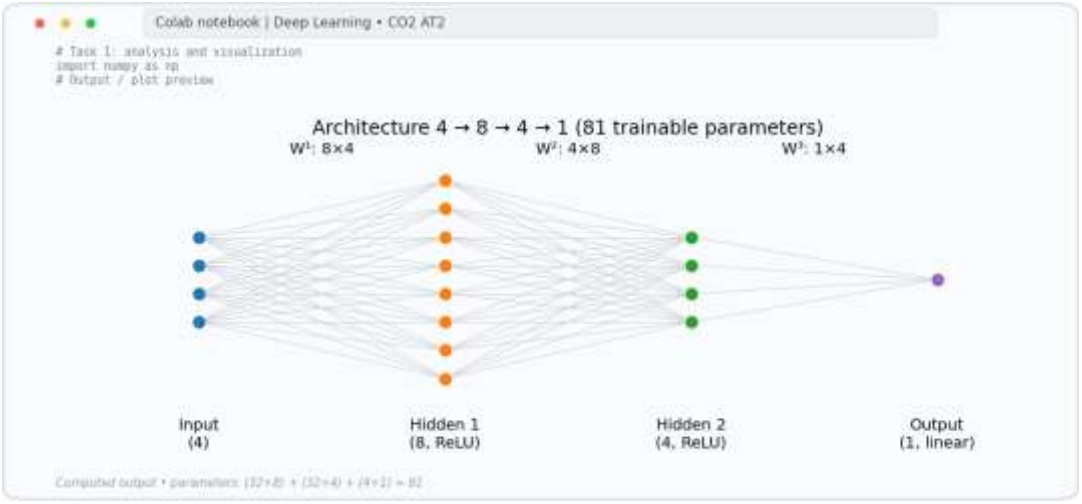
Parameter dimensions

Method and analysis. Fix the feature order, standardize with training-set statistics, then apply the three affine maps with nonlinearities between them. Without the ReLU, the three matrices would collapse into one linear map, so the nonlinearity is what gives depth any value. A row-batch library such as Keras stores kernels as $(n_{in} \times n_{out})$, i.e. the transposes, which is the same network.

Interpretation. A 4–8–4–1 network is small and suited to four tabular features; the 81 parameters limit overfitting but also limit capacity, so its validation error should be compared with a linear baseline.

Parameter	Shape	Count	Role
W(1), b(1)	8 × 4, 8 × 1	32 + 8 = 40	Inputs → hidden layer 1
W(2), b(2)	4 × 8, 4 × 1	32 + 4 = 36	Hidden 1 → hidden 2
W(3), b(3)	1 × 4, 1 × 1	4 + 1 = 5	Hidden 2 → yield
Total	—	81	All trainable parameters

```
model = keras.Sequential([layers.Input(shape=(4,)),
  layers.Dense(8, activation="relu"), layers.Dense(4, activation="relu"),
layers.Dense(1)])                    # linear output for regression
model.compile(optimizer="adam", loss="mse"); model.summary()   # 81 parameters
```



TASK 2 — Forward propagation and activation functions

The neuron receives x = (0.6, 0.8, 0.5) with weights w = (0.4, 0.3, 0.2) and bias b = 0.1. **(a)** The weighted sum (pre-activation) is $z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0.4(0.6) + 0.3(0.8) + 0.2(0.5) + 0.1 = 0.24 + 0.24 + 0.10 + 0.10 =$ **0.68**. Check: the three products add to 0.58 and the bias adds 0.10. **(b)** ReLU(z) = max(0, z) = max(0, 0.68) = **0.68**. Because z is positive, the ReLU passes the value unchanged and the neuron is "active": it sends 0.68 to the next layer, and its local derivative is 1 so gradients also pass through. Had z been negative, the output would be 0 and the neuron would be silent for that input. The first two inputs (rainfall and temperature terms) contribute the most, 0.24 each.

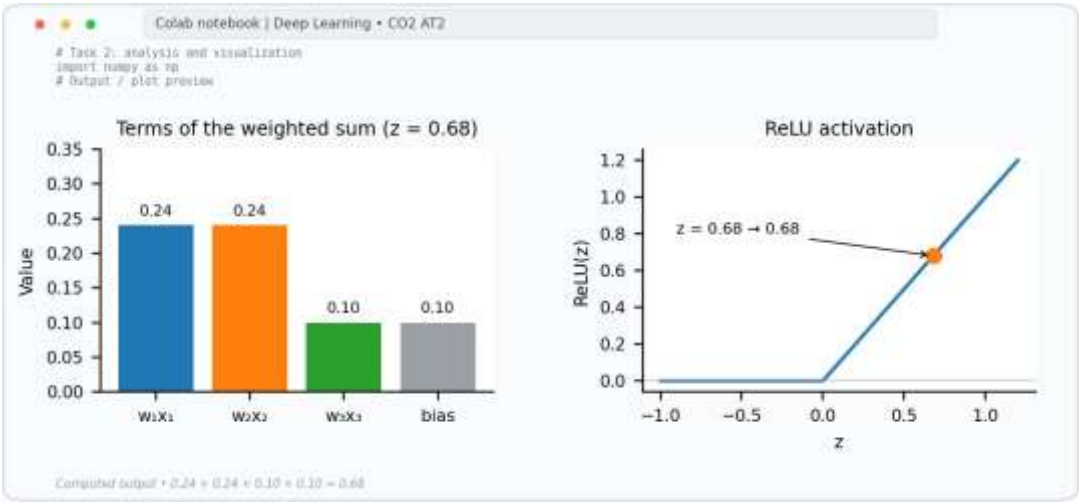
Weighted-sum calculation

Method and analysis. Multiply every input by its weight, add the products and the bias to get z, then pass z through the activation function. The activation is what makes the neuron nonlinear.

Interpretation. A positive output shows the evidence for this feature pattern outweighs the bias. ReLU is cheap and does not saturate for positive z, but a neuron whose z is always negative stops learning ("dead ReLU").

Input x	Weight w	Product wx
0.6	0.4	0.24
0.8	0.3	0.24
0.5	0.2	0.10
Bias b	—	0.10
z = Σwx + b	—	0.68 → ReLU = 0.68

```
x = np.array([0.6, 0.8, 0.5]); w = np.array([0.4, 0.3, 0.2]); b = 0.1
z = w @ x + b                  # 0.68
a = max(0.0, z)                # ReLU -> 0.68
print(round(z, 2), round(a, 2))
```



TASK 3 — Gradient-based learning and loss calculation

(a) A loss function turns "how wrong is the prediction?" into one number that training can minimize; without it there is no objective and no gradient to follow. For continuous crop yield, the squared-error (MSE) loss is suitable: it is smooth and differentiable, it penalizes large errors strongly and its minimizer is the conditional mean. MAE or Huber loss are alternatives when outliers matter. **(b)** With $y = 50$ and $\hat{y} = 42$, $L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = \mathbf{64}$. **(c)** The prediction is 8 units below the actual yield (RMSE-style error = $\sqrt{64} = 8$). The value 64 is in squared yield units, is not a percentage, and has meaning only relative to the target's scale; a prediction of 48 would give 4, a much smaller loss, while 0 means a perfect prediction. A low training loss alone does not prove good performance on unseen farms. **(d)** Gradient-based learning repeats four steps: forward pass to $\hat{y}$, loss computation, back-propagation of $\partial L/\partial\theta$ to every weight and bias, and the update $\theta \leftarrow \theta - \eta\, \partial L/\partial\theta$. Here $\partial L/\partial\hat{y} = 2(\hat{y} - y) = 2(42 - 50) = -16$; the negative sign says that increasing $\hat{y}$ would reduce the loss, and the chain rule converts this into changes for each weight.

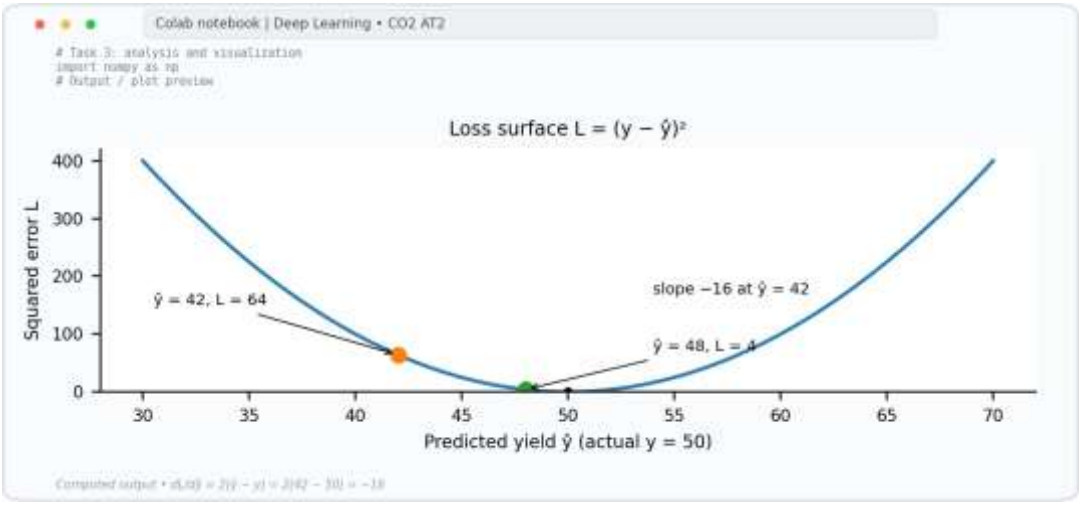
Loss for several predictions (actual y = 50)

Method and analysis. Select the loss from the target type (regression → squared error), compute it for the single example, and inspect its slope to see which direction the prediction should move. For a batch, the loss is averaged: $MSE = (1/N)\Sigma(y_i - \hat{y}_i)^2$.

Interpretation. Squared error grows quadratically: an error of 8 costs 64 but an error of 4 costs only 16, so a few large mistakes dominate the training signal. Report MAE or RMSE as well, because they are in yield units.

Prediction $\hat{y}$	Error $y - \hat{y}$	Squared error	Absolute error	$dL/d\hat{y}$
42	8	64	8	−16
48	2	4	2	−4
50	0	0	0	0
55	−5	25	5	+10

```
y, y_hat = 50.0, 42.0
loss = (y - y_hat) ** 2            # 64.0
dL_dyhat = 2 * (y_hat - y)         # -16.0
print(loss, dL_dyhat, abs(y - y_hat))
```



TASK 4 — Gradient descent and weight update

(a) The gradient-descent rule is $w_{new} = w_{old} - \eta\, \partial L/\partial w$. With $w = 0.5$, $\partial L/\partial w = 0.8$ and $\eta = 0.1$: $w_{new} = 0.5 - 0.1(0.8) = 0.5 - 0.08 =$ **0.42**. **(b)** The gradient points in the direction in which the loss increases fastest. Because the aim is to reduce the loss, the weight must move the opposite way; here the gradient is positive, so the weight decreases. The rule is only locally valid (it uses a straight-line approximation of the loss), so the step must be small enough not to jump past the minimum. **(c)** An **extremely small** η gives stable but very slow progress, requiring many epochs and risking being stuck on plateaus. An **extremely large** η overshoots the minimum, so the loss oscillates, grows, or becomes NaN. To see this, take the loss $L(w) = 0.8wZ$ (its gradient at $w = 0.5$ is 0.8): each step multiplies w by $(1 - 1.6\eta)$, so $\eta = 0.1$ shrinks w by 0.84 per step, $\eta = 0.001$ by 0.9984 (very slow) and $\eta = 1.3$ by -1.08, which flips sign and grows in magnitude, i.e. divergence.

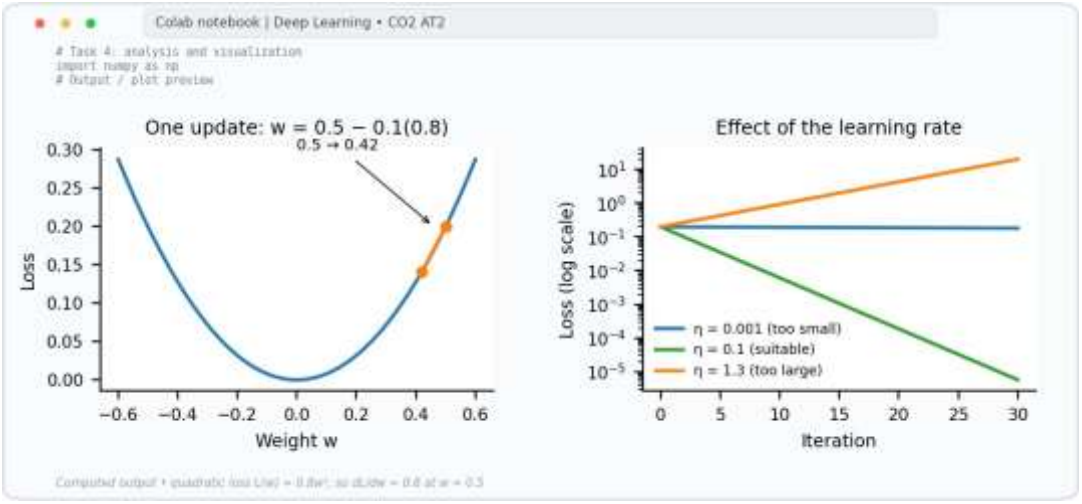
Update calculation and learning-rate comparison

Method and analysis. Compute the step size η × gradient, subtract it from the weight, and then re-evaluate the loss. On the quadratic example, L falls from $0.8(0.5)Z = 0.200$ to $0.8(0.42)Z = 0.141$ after one suitable step.

Interpretation. There is no universally best learning rate; it depends on the loss scale, batch size and optimizer, and should be tuned using validation loss, with schedules and gradient clipping as safeguards.

Learning rate η	First update from w = 0.5	Per-step factor on w	Behaviour
0.001	0.5 − 0.0008 = 0.4992	0.9984	Stable, extremely slow
0.1	0.5 − 0.08 = 0.42	0.84	Converges steadily
1.3	0.5 − 1.04 = −0.54	−1.08	Overshoots and diverges

```
w, g, lr = 0.5, 0.8, 0.1
w_new = w - lr * g              # 0.42
for lr in (0.001, 0.1, 1.3):
  w = 0.5; [w := w - lr * 1.6 * w for _ in range(30)]; print(lr, round(0.8 * w**2, 6))
```



TASK 5 — Back-propagation through multiple layers

(a) A forward pass produces $\hat{y}$ and the loss L. Back-propagation then walks backward from the output: it first computes how L changes with the output, $\partial L/\partial \hat{y}$, then the error signal $\delta^{(l)} = \partial L/\partial z^{(l)}$ of each layer, reusing the stored activations. A weight's gradient is its input activation multiplied by the error signal arriving at its neuron, so weights that feed strongly active neurons and strongly influence the output receive large gradients, i.e. a large "share of blame". **(b)** The network is a composition $\hat{y} = f_3(f_2(f_1(x)))$, so the chain rule multiplies local derivatives along the path: $\partial L/\partial W^{(l)} = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial h^{(2)})(\partial h^{(2)}/\partial h^{(1)}) \dots a^{(l-1)}$, and each layer reuses the product already computed for the layer above, which keeps the cost close to one extra forward pass. **(c)** Every weight and bias is then updated with $W \leftarrow W - \eta\, \partial L/\partial W$ and $b \leftarrow b - \eta\, \partial L/\partial b$, usually on mini-batches. Back-propagation only computes gradients; the optimizer applies them. In the 4–8–4–1 network used here the analytic gradients agree with finite-difference gradients to 6.5e-11, and the gradient norms are 1.635, 0.850 and 0.491 for $W^{(1)}$, $W^{(2)}$ and $W^{(3)}$.

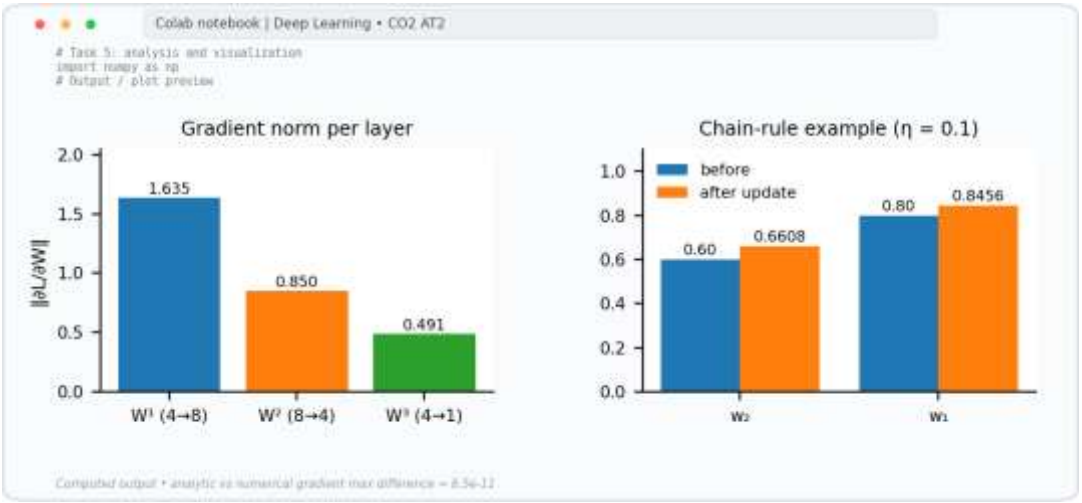
Chain-rule worked example

Method and analysis. Take a tiny chain $x = 0.5$, $h = \mathrm{ReLU}(w_1 x)$, $\hat{y} = w_2 h$ with $w_1 = 0.8$, $w_2 = 0.6$, target $y = 1$ and $\eta = 0.1$. The forward pass gives $h = 0.4$, $\hat{y} = 0.24$ and $L = 0.5776$; the backward pass and the update follow the table.

Interpretation. After one update the loss falls from 0.5776 to 0.5193 ($\hat{y} = 0.2794$), confirming that the gradients point in a useful direction. A gradient check against finite differences is a standard way to verify an implementation.

Quantity	Calculation	Value
$\partial L/\partial \hat{y}$	$2(\hat{y} - y) = 2(0.24 - 1)$	−1.52
$\partial L/\partial w_2$	$(\partial L/\partial \hat{y}) \times h = -1.52 \times 0.4$	−0.608
$\partial L/\partial h$	$(\partial L/\partial \hat{y}) \times w_2 = -1.52 \times 0.6$	−0.912
$\partial L/\partial w_1$	$(\partial L/\partial h) \times \mathrm{ReLU}'(1) \times x = -0.912 \times 1 \times 0.5$	−0.456
Updated weights	$w_2 = 0.6 + 0.0608$; $w_1 = 0.8 + 0.0456$	0.6608; 0.8456

```
x, w1, w2, y = 0.5, 0.8, 0.6, 1.0
h = max(0, w1*x); y_hat = w2*h              # 0.4, 0.24
d_y = 2*(y_hat - y); g2 = d_y*h; g1 = d_y*w2*(w1*x > 0)*x
print(g2, g1, w2 - 0.1*g2, w1 - 0.1*g1)     # -0.608 -0.456 0.6608 0.8456
```



TASK 6 — Activation functions and vanishing/exploding gradients

(a) During back-propagation the gradient reaching an early layer contains a product of one activation derivative and one weight matrix per layer above it. If the derivatives are well below 1, repeated multiplication shrinks the gradient towards zero (vanishing), so early layers learn very slowly; if the factors are above 1, the product grows (exploding) and updates become unstable. Sigmoid is the clearest cause: its derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ is at most 0.25, so ten layers can scale the gradient by at most $0.25^{10} \approx 9.5 \times 10^{-7}$. In the simulation of a 10-layer network the sigmoid gradient norm is 3.0e-07 at layer 1 against 0.95 at layer 10, whereas tanh gives 0.20 and 0.31 and ReLU 1.31 and 8.03. **(b)** Sigmoid (range 0–1) is not zero-centred and saturates; tanh (range −1 to 1) is zero-centred with maximum derivative 1 but also saturates; ReLU has derivative exactly 1 for positive inputs and 0 for negative ones, is cheap, and does not saturate on the positive side. **(c)** Hence ReLU (or Leaky ReLU) with He initialization is the default for deep hidden layers, while sigmoid is reserved for binary outputs and softmax for multi-class outputs. Activation choice must be combined with good initialization, batch normalization, residual connections and gradient clipping to control exploding gradients.

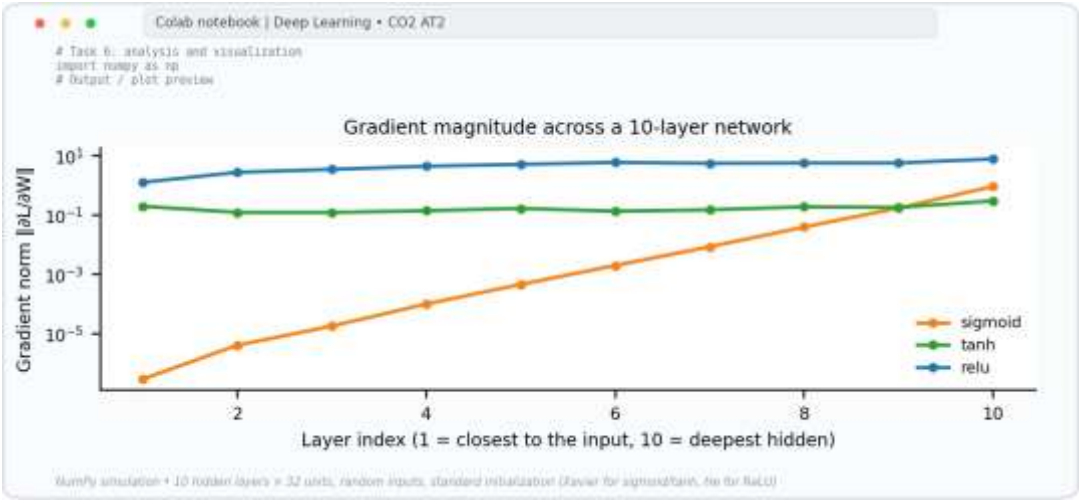
Activation comparison

Method and analysis. A 10-hidden-layer, 32-unit network is back-propagated once for each activation, and the gradient norm of every layer's weights is recorded on a logarithmic scale.

Interpretation. With Xavier initialization tanh did not vanish in this small test, so initialization matters as much as the function; ReLU gradients did not shrink toward the input and can even grow, which is why clipping is kept available.

Activation	Range / derivative	Strength	Limitation
Sigmoid	(0, 1); $\sigma' \leq 0.25$	Probability output	Saturates, vanishing gradients
tanh	(−1, 1); $\tanh' \leq 1$	Zero-centred	Saturates for large $\lvert z \rvert$
ReLU	[0, ∞); 0 or 1	Fast, no positive saturation	Dead units for $z < 0$

```
for act in ("sigmoid", "tanh", "relu"):
  net = MLP([32]*11 + [1], act=act)          # 10 hidden layers
  norms = layer_gradient_norms(net, X)       # back-propagate once
  print(act, norms[0], norms[-1])
```



TASK 7 — Back-propagation and optimization strategy

(a) The learning rate η scales every parameter update. A small η gives a small, safe step but slow loss reduction; a large η moves quickly at first but can bounce across the valley, producing a noisy or diverging loss. Mini-batch gradients are also noisy, so a large η amplifies this noise. **(b)** One iteration is: take a mini-batch; forward pass to predictions; compute the loss (plus any regularization term); back-propagation computes $\partial L/\partial \theta$ with the chain rule; the optimizer updates $\theta \leftarrow \theta - \eta \partial L/\partial \theta$ (or momentum/Adam variants); and loss values are logged. Back-propagation supplies the gradients and gradient descent consumes them. **(c)** For slow decrease: check feature and target scaling, increase η cautiously (for example from 0.001 to 0.03), add momentum or use Adam, verify initialization and gradients. For fast but fluctuating loss: lower η or decay it, use momentum or Adam with tuned settings, enlarge the batch, add gradient clipping and check noisy labels. **(d)** The training curve shows whether optimization works; the validation curve shows whether progress generalizes. Divergence, plateaus, oscillation and a widening train–validation gap are all visible, and the best-validation checkpoint can be kept with early stopping. In the simulation, η = 0.001 only reaches a loss of 0.787 after 60 epochs, η = 0.03 reaches 0.144, η = 0.2 is fast early (0.199 at epoch 10) but fluctuates (final 0.670), and η = 0.2 with step decay settles at 0.087.

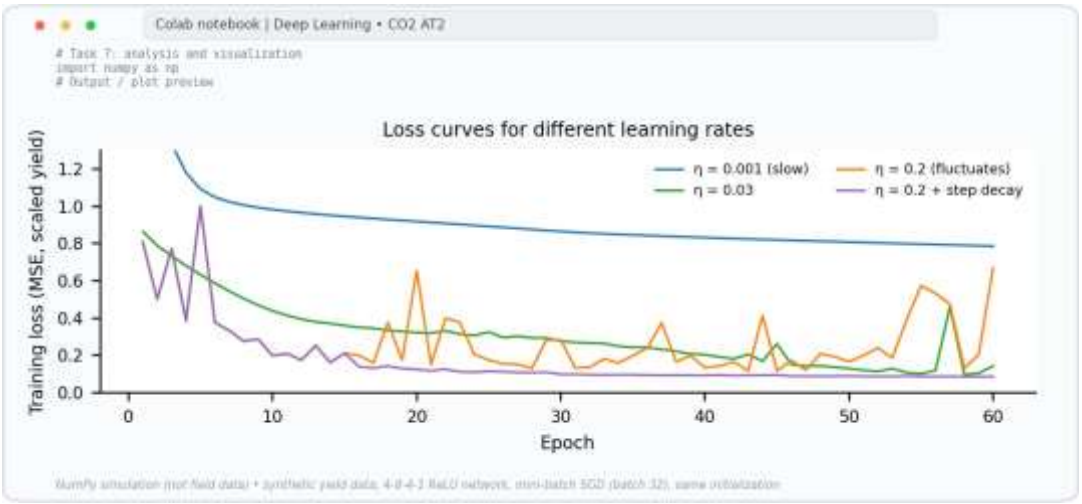
Loss-curve diagnosis

Method and analysis. The same network, data and initial weights are trained for 60 epochs with four learning-rate settings; training loss is recorded after every epoch.

Interpretation. The average epoch-to-epoch change in loss was 0.098 for the fluctuating run and only 0.0022 after step decay, showing that a schedule cures oscillation without losing the early speed.

Experiment	Setting	Loss at epoch 10	Loss at epoch 60	Diagnosis / fix
Slow	η = 0.001	0.982	0.787	Too small: raise η, use Adam
Steady	η = 0.03	0.439	0.144	Good baseline
Fluctuating	η = 0.2	0.199	0.670	Too large: decay or lower η
Fixed	η = 0.2, halved every 15 epochs	0.199	0.087	Fast and stable

```
for lr in (0.001, 0.03, 0.2):
  net = MLP([4, 8, 4, 1], seed=3)            # identical initialization
  hist = train(net, X_tr, y_tr, X_va, y_va, epochs=60, lr=lr, batch=32)
  plt.plot(hist[:, 0], label=f"lr={lr}")
```



TASK 8 — Regularization and overfitting control

(a) Training accuracy of 99% against validation accuracy of 78% is a gap of 21 percentage points: the model has memorized the training examples (low bias, high variance), which is **overfitting**. Possible confounders such as leakage, a small or unrepresentative validation set and distribution shift should also be checked. **(b)** L1 regularization adds $\lambda\Sigma|w_j|$, which pushes many weights exactly to zero and yields sparse, simpler models; L2 adds $\lambda\Sigma w_j^2$, which shrinks all weights smoothly (weight decay) and prevents any single weight from becoming very large. Both limit the effective complexity of the network. **(c)** The L2-regularized loss is $L_{total} = (1/N)\Sigma(y_i - \hat{y}_i)^2 + \lambda\Sigma_j w_j^2$, with gradient $2\lambda w_j$ added to each weight's gradient. The parameter $\lambda \geq 0$ sets the trade-off: $\lambda = 0$ is no regularization, a very large λ forces weights to zero and underfits. For example, $w = (2, -1, 0.5)$ and $\lambda = 0.1$ give an L2 penalty of $0.1(4 + 1 + 0.25) = 0.525$ and an L1 penalty of $0.1(3.5) = 0.35$. **(d)** Dropout randomly zeroes a fraction of hidden activations in each training step, so units cannot rely on one another (co-adaptation) and the network behaves like an ensemble of thinned networks; at inference all units are used with scaled activations. In the simulation (60 training points, 4–32–32–1 network) L2 with $\lambda = 0.001$ reduced validation MSE from 0.128 to 0.085, whereas dropout 0.2 did not help here (0.161), a reminder that every regularizer must be validated.

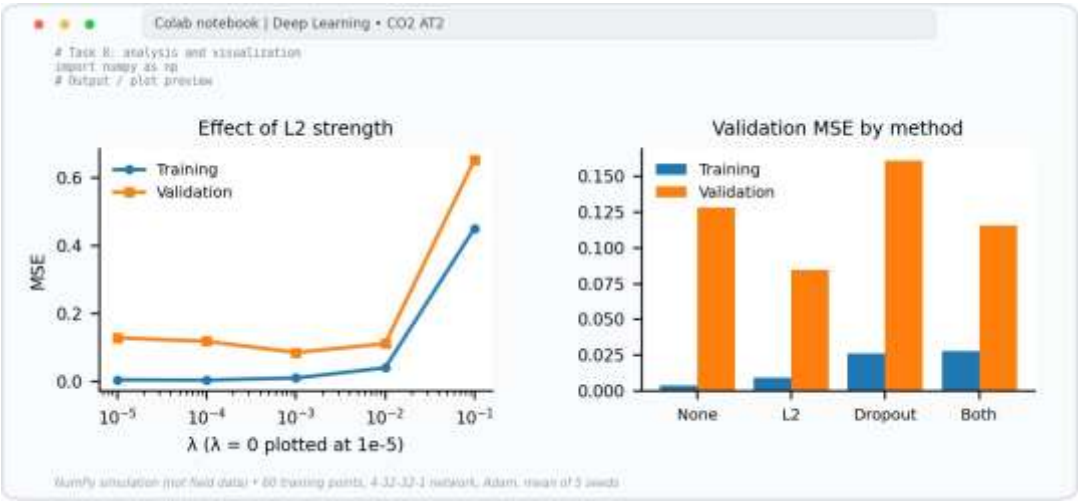
Regularization experiment

Method and analysis. The same over-parameterized network is trained on a small noisy sample with and without regularization, averaged over five seeds, and training and validation MSE are compared.

Interpretation. Regularization raises training error slightly but lowers validation error; too large a λ (0.1) underfits badly (validation MSE 0.652). Early stopping, augmentation and more data are further remedies.

Method	Training MSE	Validation MSE	Comment
No regularization	0.004	0.128	Overfits
L2 ($\lambda = 0.001$)	0.009	0.085	Best validation
Dropout (0.2)	0.026	0.161	No gain here
L2 + dropout	0.028	0.115	Better than none

```
layers.Dense(32, activation="relu", kernel_regularizer=regularizers.l2(1e-3)),
layers.Dropout(0.2),                    # active only during training
layers.Dense(32, activation="relu", kernel_regularizer=regularizers.l2(1e-3)),
layers.Dense(1)
```



TASK 9 — Convolution operation and feature extraction

(a) Disease signs such as spots, edges and texture are local patterns that can appear anywhere on a leaf. A convolutional layer slides a small learnable filter over the image, so it looks at neighbouring pixels together (local connectivity), reuses the same weights at every position (weight sharing, hence far fewer parameters than a dense layer on all pixels) and detects a pattern wherever it occurs (translation equivariance). **(b)** Take a 5 × 5 grayscale leaf patch with a dark 2 × 2 lesion (value 2) on a bright background (value 8) and the vertical-edge filter K = [[1, 0, −1], [1, 0, −1], [1, 0, −1]]. The first window is [[8,8,8],[8,2,2],[8,2,2]] and the output is (8−8) + (8−2) + (8−2) = 12. Moving the filter over all nine positions (stride 1, no padding) gives the 3 × 3 feature map [[12, −12, −12], [12, −12, −12], [6, −6, −6]]. Positive values mark bright-to-dark transitions (left edge of the lesion) and negative values mark dark-to-bright transitions (right edge); a uniform background would give 0. **(c) Filters** are the learned weights that decide which pattern is detected, and each filter produces one feature map. **Stride** is the step of the window: stride 2 on this example yields a 2 × 2 map [[12, −12], [6, −6]], reducing computation but losing detail. **Padding** adds border zeros: padding 1 keeps the output 5 × 5 and lets border pixels be processed. The output size is floor((N + 2P – F)/S) + 1.

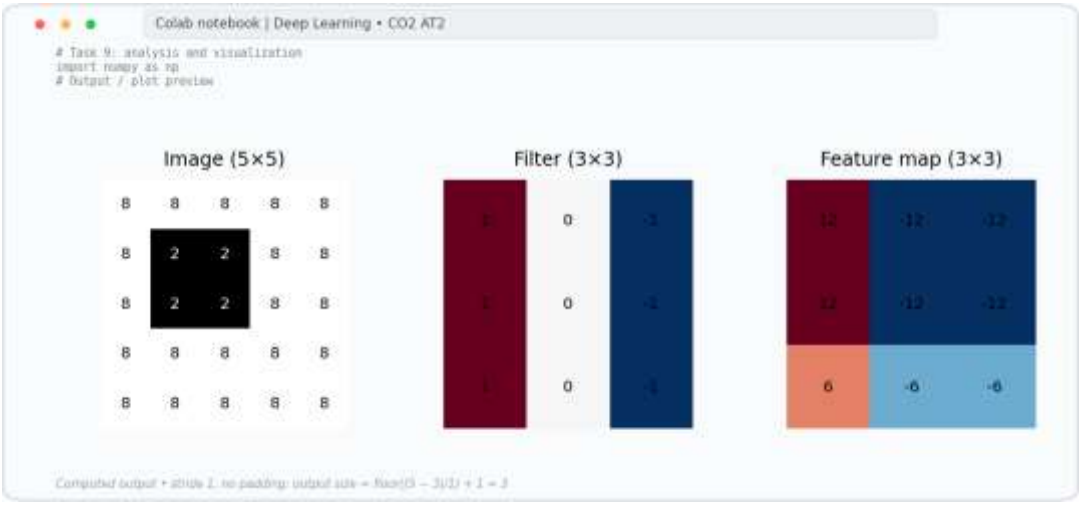
Convolution worked example

Method and analysis. At every window, multiply the filter and the image patch element by element and add the products (cross-correlation, the operation implemented as “convolution” in deep-learning libraries).

Interpretation. This hand-made filter shows the arithmetic; in a CNN the filter values are learned from labelled leaf images by back-propagation, and stacked filters respond to spots, vein edges and textures.

Setting	Value in the example	Output size	Effect
Filter 3 × 3	Vertical-edge weights	—	Detects left/right edges
Stride 1, padding 0	floor((5 − 3)/1) + 1	3 × 3	Full detail, smaller map
Stride 2, padding 0	floor((5 − 3)/2) + 1	2 × 2	Less computation, coarser
Stride 1, padding 1	floor((5 + 2 − 3)/1) + 1	5 × 5	Size preserved

```
I = np.array([[8,8,8,8,8],[8,2,2,8,8],[8,2,2,8,8],[8,8,8,8,8],[8,8,8,8,8]])
K = np.array([[1,0,-1]]*3)
out = np.array([[(I[i:i+3, j:j+3] * K).sum() for j in range(3)] for i in range(3)])
print(out)                    # [[12 -12 -12] [12 -12 -12] [6 -6 -6]]
```



TASK 10 — CNN architecture and pooling

(a) Input image: a pixel tensor (height × width × channels). **Convolution:** learned filters produce feature maps of local patterns. **ReLU:** adds nonlinearity so that stacked layers are not equivalent to one linear filter. **Pooling:** summarizes small regions, shrinking the maps and giving tolerance to small shifts. The second convolution–ReLU–pooling block combines the first block's simple patterns into richer ones. The **fully connected layer** combines the flattened features, and the **output layer** gives class scores (softmax for healthy/diseased). **(b)** A 2 × 2 max-pool with stride 2 keeps the largest value in each non-overlapping block, so a 4 × 4 map becomes 2 × 2 and the number of values falls by a factor of 4. For [[2,4,1,0],[5,3,2,6],[1,0,7,8],[4,2,3,5]] the result is [[5, 6], [4, 8]]; average pooling would give [[3.5, 2.25], [1.75, 5.75]]. Pooling has no learned weights. **(c)** Larger filters see more context but need more weights and computation; a larger stride shrinks the feature map and cost but may skip small lesions; more filters create more output channels (more feature types) but increase parameters and memory. For example a 32 × 32 input with a 3 × 3 filter, padding 1 and stride 1 stays 32 × 32 (32 × 32 × 8 with 8 filters), while stride 2 gives 16 × 16. **(d)** Early layers learn edges and colour contrasts, middle layers combine them into spots, veins and lesion boundaries, and deeper layers combine those into disease-specific structures, so each layer's receptive field grows and its features become more abstract.

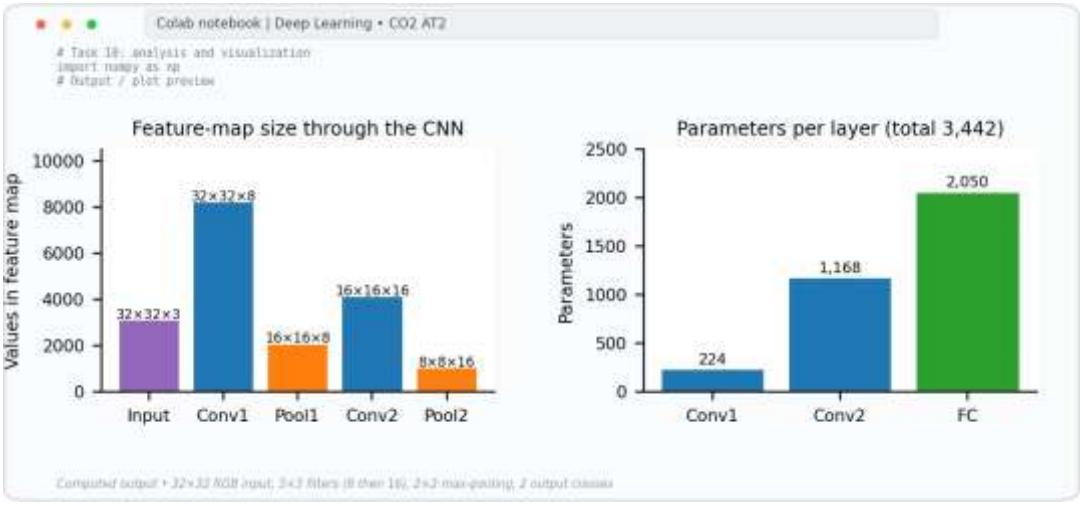
Example CNN dimensions (32 × 32 RGB leaf, 2 classes)

Method and analysis. Output size = floor((N + 2P − F)/S) + 1 is applied layer by layer; parameters of a convolution are $F \times F \times C_{in} \times C_{out} + C_{out}$.

Interpretation. The CNN has only 3,442 parameters, whereas one dense layer of 128 units on the raw 32 × 32 × 3 pixels already needs 393,344. Deeper networks are not always better; small data sets may favour transfer learning.

Stage	Operation	Output shape	Parameters
Input	RGB image	32 × 32 × 3	0
Conv1 + ReLU	8 filters 3 × 3, stride 1, pad 1	32 × 32 × 8	224
Pool1	2 × 2 max, stride 2	16 × 16 × 8	0
Conv2 + ReLU	16 filters 3 × 3, pad 1	16 × 16 × 16	1,168
Pool2	2 × 2 max, stride 2	8 × 8 × 16	0
Flatten + FC	1,024 → 2 (softmax)	2	2,050

```
cnn = keras.Sequential([layers.Input(shape=(32,32,3)),
  layers.Conv2D(8, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
  layers.Conv2D(16, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
  layers.Flatten(), layers.Dense(2, activation="softmax")]); cnn.summary()   # 3,442
```



TASK 11 — Comparative evaluation of feed-forward and CNN models

(a) Rainfall, temperature, soil moisture and fertilizer quantity are a fixed-length vector of independent numeric features with no spatial arrangement. A feed-forward network (MLP) accepts exactly such a vector, learns nonlinear interactions through dense layers, and with a linear output predicts a continuous yield. It is small (81 parameters for 4–8–4–1) and fast to train. **(b)** A leaf image has a two-dimensional structure in which neighbouring pixels form edges, spots and textures. A CNN uses local filters with shared weights, pooling and a hierarchy of feature maps, so it exploits this structure with relatively few parameters and detects lesions at any position. **(c)** See the comparison table: the MLP takes a flat numeric vector and uses dense matrices in which every weight is used once per sample, while the CNN takes an H × W × C tensor and reuses small kernels at all locations. A dense layer on 224 × 224 × 3 pixels would already need 19,267,712 parameters for 128 units, and the example CNN needs 3,442 parameters but about 0.52 million multiply–adds per 32 × 32 image against 81 for the yield MLP, so CNN cost scales with resolution, channels and depth. **(d)** Recommendation: use the **MLP for crop-yield regression** (structured data, small model, MAE/RMSE/RZ evaluation) and the **CNN for leaf-disease classification** (spatial data, accuracy/macro-F1/recall evaluation), preferably with transfer learning. The two models should not be forced into one architecture, and their errors must not be compared directly.

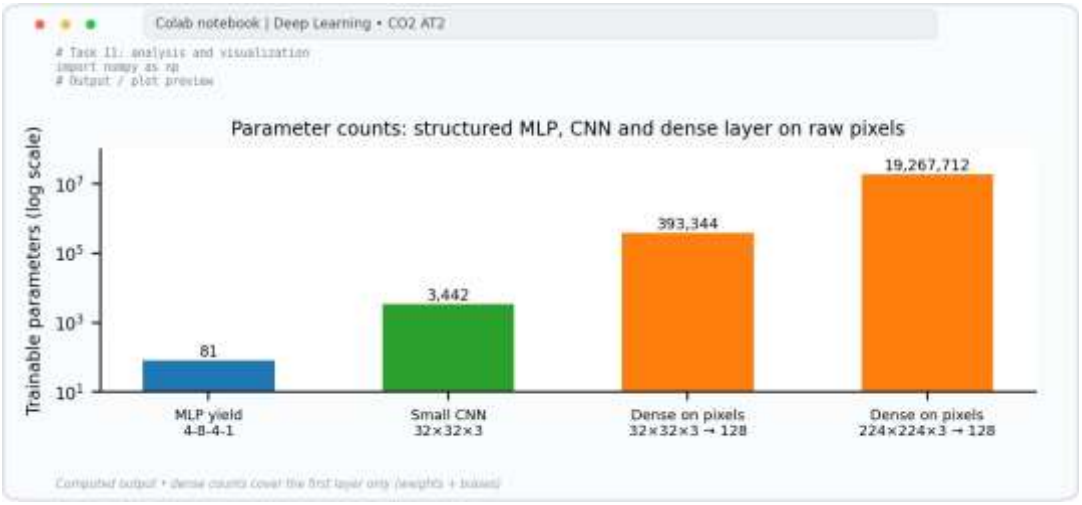
Technical comparison

Method and analysis. The models are compared by input representation, feature extraction, parameter organization and computation, using exact parameter counts and multiply–add counts calculated from the architectures above.

Interpretation. Data type should drive architecture. Using an MLP on pixels wastes parameters and ignores neighbourhood structure; using a CNN on four tabular numbers has nothing spatial to exploit.

Dimension	Model A: feed-forward (yield)	Model B: CNN (leaf images)
Input	Vector of 4 normalized numbers	Image tensor H × W × 3
Feature extraction	Dense nonlinear combinations	Learned local filters, hierarchy of maps
Parameter organization	Full matrices 8×4, 4×8, 1×4 (81)	Shared kernels + small classifier (3,442)
Computation	≈ 81 multiply–adds per record	≈ 0.52 million per 32 × 32 image; grows with size
Output / metrics	Continuous yield; MAE, RMSE, RZ	Class probabilities; accuracy, macro-F1

```
mlp_params = 4*8+8 + 8*4+4 + 4*1+1                  # 81
cnn_macs = 32*32*8*27 + 16*16*16*72 + 1024*2        # 518,144
dense_pixels = 224*224*3*128 + 128                  # 19,267,712
print(mlp_params, cnn_macs, dense_pixels)
```



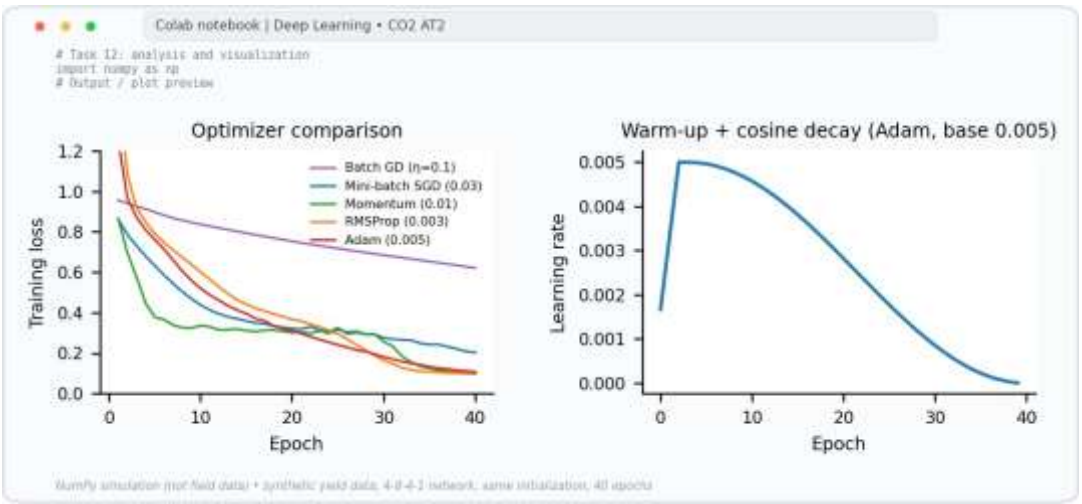
TASK 12 — Design challenge: optimize and redesign the deep learning system

Decision: redesign into two task-specific branches, keeping the original 4–8–4–1 network only as a baseline. **A. Impact analysis. (a)** 500,000 records are 50 times the original 10,000. With four float32 features the raw matrix is only 500,000 × 4 × 4 B = 8 MB, so the numerical branch is cheap in memory, but 50 times more updates per epoch and hyper-parameter searches make training time and mini-batching important. A 224 × 224 × 3 uint8 image needs 150,528 B; 100,000 images need about 15.1 GB, and about 60.2 GB as float32, so images must be streamed in batches from disk, with parallel decoding, caching and mixed precision, and GPU compute is needed. **(b)** A feed-forward network applied to flattened pixels discards the 2-D neighbourhood structure, has no weight sharing or translation tolerance, and a single dense layer of 128 units on 224 × 224 × 3 pixels has 19.27 million parameters, which overfits and is slow. **B. Existing design. (a)** High training but lower validation performance means high variance (overfitting) or a train–validation mismatch; check leakage, duplicates, labels and farm/season splits first. **(b)** Simply adding layers and neurons raises capacity, training time and overfitting risk; it is not necessarily the best fix. Better data coverage, regularization, early stopping, a suitable architecture and validated hyper-parameters should come first.

C. Redesign. (a) Numerical branch: 4 normalized inputs → 64 → 32 → 16 ReLU layers (compared against 4–8–4–1) → 1 linear output, MSE/Huber loss, L2 weight decay, early stopping, Adam. **Image branch:** 224 × 224 × 3 input, pretrained EfficientNet-B0 or ResNet-18 backbone, global average pooling, dropout, softmax head, cross-entropy loss, transfer learning followed by fine-tuning. **(b)** Optimizers on the same synthetic regression task (training loss after 40 epochs): batch GD 0.623, mini-batch SGD 0.205, momentum 0.100, RMSProp 0.103 and Adam 0.108, so adaptive and momentum methods converge fastest per epoch. Use Adam ($\eta = 0.005$ for the MLP, 1e−4 for fine-tuning) with a 3-epoch warm-up and cosine decay, batch normalization in the CNN, ReLU hidden activations and gradient clipping if instability appears.

Design element	Numerical branch (MLP)	Leaf-image branch (CNN)
Input	4 standardized features	224 × 224 × 3, ImageNet normalization
Architecture	64–32–16–1, ReLU, linear output	EfficientNet-B0 / ResNet-18 + pooled head
Loss and metrics	MSE or Huber; MAE, RMSE, RZ	Cross-entropy; accuracy, macro-F1, recall
Regularization	L2, early stopping, dropout if validated	Augmentation, dropout, weight decay, BN
Optimizer	Adam, warm-up + cosine decay	Adam / SGD + momentum, smaller fine-tune η

```
base = keras.applications.EfficientNetB0(include_top=False, weights="imagenet", pooling="avg")
sched = keras.optimizers.schedules.CosineDecay(5e-3, decay_steps=total_steps)
model.compile(optimizer=keras.optimizers.Adam(sched), loss="mse")        # yield branch
early = keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)
```



D. Evidence-based defence. (a) Experiment: use leakage-safe splits (by farm/season for yield, by plant/source for images) with an untouched test set. Compare, for yield, the original 4–8–4–1, the regularized MLP and a linear baseline using MAE, RMSE, RZ, validation loss and training time per epoch; for images compare a small CNN, ResNet-18 and EfficientNet-B0 using accuracy, macro-F1, per-class recall, confusion matrix, validation loss, training time, memory and inference latency; repeat with several seeds and run ablations of optimizer, learning rate, augmentation, dropout and weight decay. **(b) Decision:** redesign, because the observed gap, the 50-fold data growth and the new image modality are all structural problems. The original model is retained as a baseline and is kept for yield prediction only if the experiment shows it matches the new MLP.

Image augmentation and architectures. Label-preserving augmentation applied to training images only: small rotations (±20°), horizontal and vertical flips (leaves have no canonical orientation), ±10% translation and zoom, brightness/contrast ±15% and slight colour jitter; avoid heavy crops or colour shifts that remove lesions or imitate another disease. For a 224 × 224 input and a limited compute budget, EfficientNet-B0 (about 5.3 M parameters) or ResNet-18 (11.7 M) with transfer learning is preferred, LeNet-5 (about 0.06 M) suits only tiny 32 × 32 images, and AlexNet (about 61 M) is heavy for its accuracy.

Network	Input size	Parameters (approx.)	Suitability for 100,000 leaf images
LeNet-5	32 × 32	0.06 M	Too small for varied leaf images
AlexNet	227 × 227	61 M	Heavy dense layers; superseded
ResNet-18 / 50	224 × 224	11.7 M / 25.6 M	Strong, stable baseline (skip connections)
EfficientNet-B0	224 × 224	5.3 M	Best accuracy per compute; recommended

Optimizer comparison (final training loss from the NumPy simulation, 40 epochs):

Optimizer	Idea	Loss	When to use
Batch GD	All records per update; smooth but slow	0.623	Reference only; too costly at 500,000 records
Mini-batch SGD	Small random batch; noisy, scalable	0.205	Strong baseline with a tuned schedule
Momentum	Accumulates past gradients	0.100	Often best for CNNs with a decay schedule
RMSProp	Scales steps by recent squared gradients	0.103	Differently scaled or non-stationary gradients
Adam	Momentum + adaptive step sizes	0.108	Fast default for the MLP and for fine-tuning